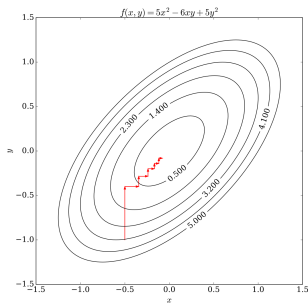
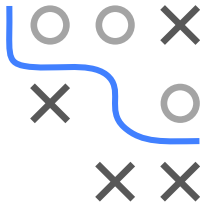


Optimization in Machine Learning

Coordinate descent

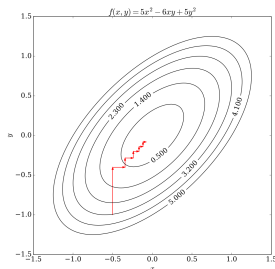
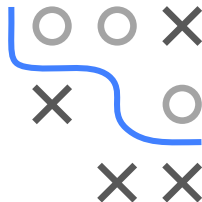


Learning goals

- Axes as descent direction
- CD on linear model and LASSO
- Soft thresholding

COORDINATE DESCENT

- **Assumption:** Objective function not differentiable
- **Idea:** Instead of gradient, use coordinate directions for descent
- First: Select starting point $\mathbf{x}^{[0]} = (x_1^{[0]}, \dots, x_d^{[0]})$
- Step t : Minimize f along x_i for each dimension i for fixed $x_1^{[t]}, \dots, x_{i-1}^{[t]}$ and $x_{i+1}^{[t-1]}, \dots, x_d^{[t-1]}$.



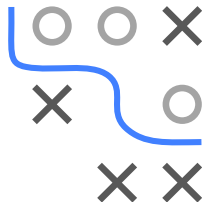
Source: Wikipedia (Coordinate descent)

COORDINATE DESCENT

- Minimum is determined with (exact / inexact) line search
- Order of dimensions can be any permutation of $\{1, 2, \dots, d\}$
- **Convergence:**
 - f convex differentiable
 - f sum of convex differentiable and *convex separable* function:

$$f(\mathbf{x}) = g(\mathbf{x}) + \sum_{i=1}^d h_i(x_i),$$

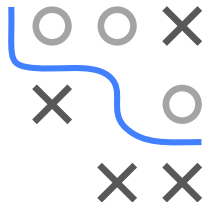
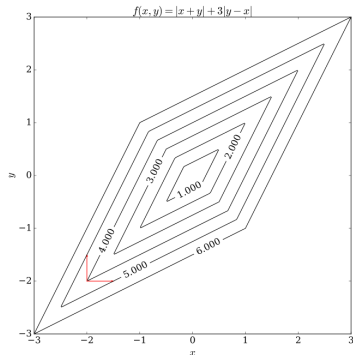
where g convex differentiable and h_i convex



COORDINATE DESCENT

Not convergence in general for convex functions.

Counterexample:



Source: Wikipedia (Coordinate descent)

EXAMPLE: LINEAR REGRESSION

Minimize LM with L2-loss via CD:

$$\min_{\theta} g(\theta) = \min_{\theta} \frac{1}{2} \sum_{i=1}^n \left(y^{(i)} - \theta^{\top} \mathbf{x}^{(i)} \right)^2 = \min_{\theta} \frac{1}{2} \|\mathbf{y} - \mathbf{X}\theta\|^2$$

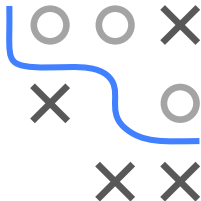
where $\mathbf{y} \in \mathbb{R}^n$, $\mathbf{X} \in \mathbb{R}^{n \times d}$ with columns $\mathbf{x}_1, \dots, \mathbf{x}_d \in \mathbb{R}^n$.

Assume: Scaled data, i.e., $\mathbf{X}^{\top} \mathbf{X} = I_d$ (just to get intuition)

Then:

$$\begin{aligned} g(\theta) &= \frac{1}{2} \mathbf{y}^{\top} \mathbf{y} + \frac{1}{2} \theta^{\top} \theta - \mathbf{y}^{\top} \mathbf{X} \theta \\ &\stackrel{(*)}{=} \frac{1}{2} \mathbf{y}^{\top} \mathbf{y} + \frac{1}{2} \theta^{\top} \theta - \mathbf{y}^{\top} \sum_{k=1}^d \mathbf{x}_k \theta_k \end{aligned}$$

$$(*) \quad \mathbf{X} \theta = \mathbf{x}_1 \theta_1 + \mathbf{x}_2 \theta_2 + \dots + \mathbf{x}_d \theta_d = \sum_{k=1}^d \mathbf{x}_k \theta_k$$



EXAMPLE: LINEAR REGRESSION

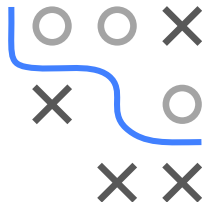
- Exact CD update in direction j :

$$\frac{\partial g(\boldsymbol{\theta})}{\partial \theta_j} = \theta_j - \mathbf{y}^\top \mathbf{x}_j$$

- By solving $\frac{\partial g(\boldsymbol{\theta})}{\partial \theta_j} = 0$, we get

$$\theta_j^* = \mathbf{y}^\top \mathbf{x}_j$$

- Repeat** this update for all θ_j



SOFT THRESHOLDING

Minimize LM with L2-loss and L1 regularization via CD:

$$\min_{\theta} h(\theta) = \min_{\theta} \frac{1}{2} \|\mathbf{y} - \mathbf{X}\theta\|^2 + \lambda \|\theta\|_1$$

Note that $h(\theta) = \frac{1}{2} \mathbf{y}^\top \mathbf{y} + \frac{1}{2} \theta^\top \theta - \sum_{k=1}^d (\mathbf{y}^\top \mathbf{x}_k \theta_k + \lambda |\theta_k|)$

Assume (again): $\mathbf{X}^\top \mathbf{X} = I_d$.

Since $|\cdot|$ is not differentiable, distinguish three cases:

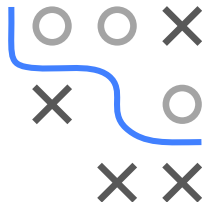
- **Case 1:** $\theta_j > 0$. Then $|\theta_j| = \theta_j$ and

$$0 = \frac{\partial h(\theta)}{\partial \theta_j} = \theta_j - \mathbf{y}^\top \mathbf{x}_j + \lambda \quad \Leftrightarrow \quad \theta_{j,\text{LASSO}}^* = \theta_j^* - \lambda$$

- **Case 2:** $\theta_j < 0$. Then $|\theta_j| = -\theta_j$ and

$$0 = \frac{\partial h(\theta)}{\partial \theta_j} = \theta_j - \mathbf{y}^\top \mathbf{x}_j - \lambda \quad \Leftrightarrow \quad \theta_{j,\text{LASSO}}^* = \theta_j^* + \lambda$$

- **Case 3:** $\theta_j = 0$



SOFT THRESHOLDING

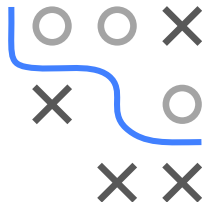
We can write the solution as:

$$\theta_{j,\text{LASSO}}^* = \begin{cases} \theta_j^* - \lambda & \text{if } \theta_j^* > \lambda \\ \theta_j^* + \lambda & \text{if } \theta_j^* < -\lambda \\ 0 & \text{if } \theta_j^* \in [-\lambda, \lambda], \end{cases}$$

This operation is called **soft thresholding**.

Coefficients for which the solution to the unregularized problem are smaller than a threshold, $|\theta_j^*| < \lambda$, are shrunk to zero.

Note: Derivation of soft thresholding operator not trivial (subgradients)



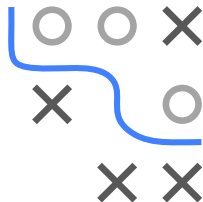
CD FOR STATISTICS AND ML

Why is it being used?

- Easy to implement
- Scalable: no storage/operations on large objects, just current point
⇒ Good implementation can achieve state-of-the-art performance
- Applicable for non-differentiable (but convex separable) objectives

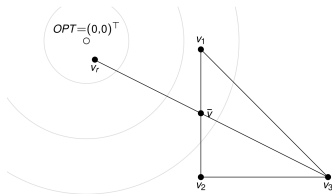
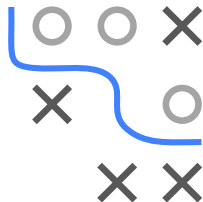
Examples:

- Lasso regression, Lasso GLM, graphical Lasso
- Support Vector Machines
- Regression with non-convex penalties



Optimization in Machine Learning

Nelder-Mead method

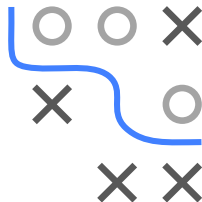


Learning goals

- General idea
- Reflection, expansion, contraction
- Advantages & disadvantages
- Examples

NELDER-MEAD METHOD

- Derivative-free method $\Rightarrow$ heuristic
- Generalization of bisection in d -dimensional space
- Based on d -simplex, defined by $d + 1$ points:
 - $d = 1$ interval
 - $d = 2$ triangle
 - $d = 3$ tetrahedron
 - ...



NELDER-MEAD METHOD

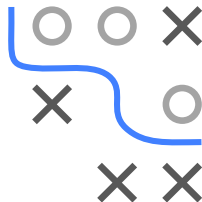
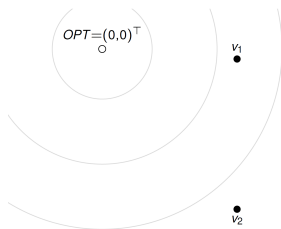
A version of the **Nelder-Mead** method:

Initialization: Choose $d + 1$ random, affinely independent points $\mathbf{v}_i$ ($\mathbf{v}_i$ are vertices: corner points of the simplex/polytope).

❶ **Order:** Order points according to ascending function values

$$f(\mathbf{v}_1) \leq f(\mathbf{v}_2) \leq \dots \leq f(\mathbf{v}_d) \leq f(\mathbf{v}_{d+1}).$$

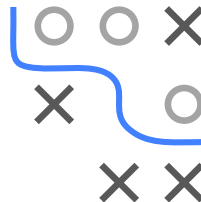
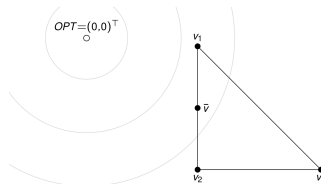
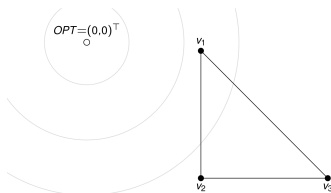
with $\mathbf{v}_1$ best point, $\mathbf{v}_{d+1}$ worst point.



NELDER-MEAD METHOD

② Compute **centroid** without worst point

$$\bar{\mathbf{v}} = \frac{1}{d} \sum_{i=1}^d \mathbf{v}_i.$$

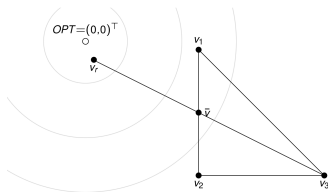


NELDER-MEAD METHOD

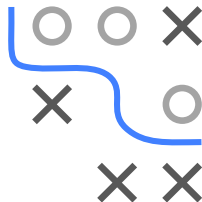
③ **Reflection:** Compute reflection point

$$\mathbf{v}_r = \bar{\mathbf{v}} + \rho(\bar{\mathbf{v}} - \mathbf{v}_{d+1}),$$

with $\rho > 0$. Compute $f(\mathbf{v}_r)$.



Note: Default value for reflection coefficient: $\rho = 1$



NELDER-MEAD METHOD

- **Case 3:** $f(\mathbf{v}_r) \geq f(\mathbf{v}_d)$

⇒ **Contraction:**

$$\mathbf{v}_c = \bar{\mathbf{v}} + \gamma(\mathbf{v}_{d+1} - \bar{\mathbf{v}})$$

with $0 < \gamma \leq 1/2$.

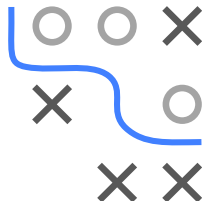
- If $f(\mathbf{v}_c) < f(\mathbf{v}_{d+1})$, accept $\mathbf{v}_c$.
- Otherwise, shrink **entire** simplex (**Shrinking**):

$$\mathbf{v}_i = \mathbf{v}_1 + \sigma(\mathbf{v}_i - \mathbf{v}_1) \quad \forall i$$

Note: Default values for contraction and shrinking coefficient:

$$\gamma = \sigma = 1/2$$

- ④ **Repeat** all steps until stopping criterion met.



NELDER-MEAD

Advantages:

- No gradients needed
- Robust, often works well for non-differentiable functions.

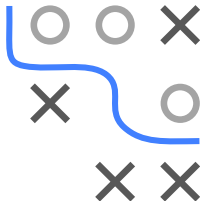
Drawbacks:

- Relatively slow (not applicable in high dimensions)
- Not each step improves solution, only mean of corner values is reduced.
- No guarantee for convergence to local optimum / stationary point.

Visualization:

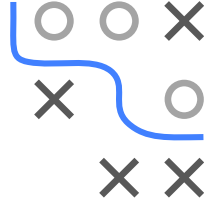
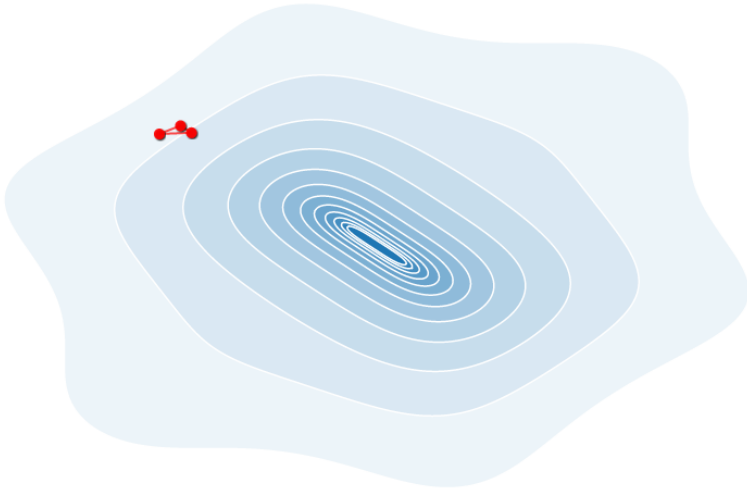
<http://www.benfrederickson.com/numerical-optimization/>

Note: Nelder-Mead is default method of R function `optim()`. If gradient is available and cheap, L-BFGS is preferred.



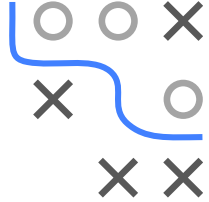
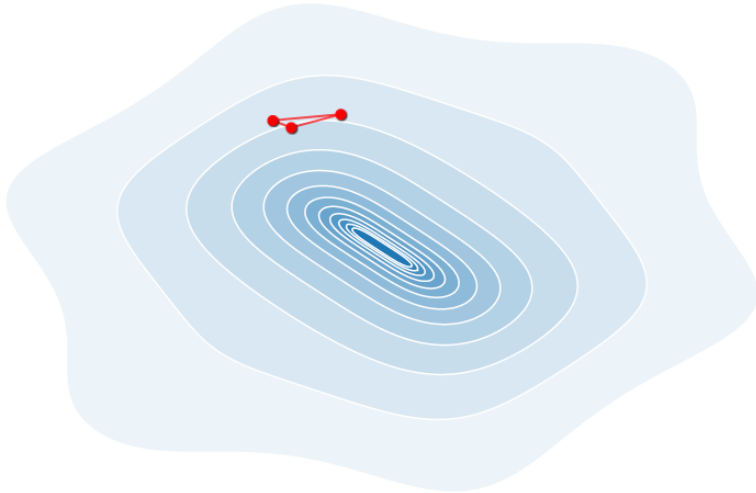
NELDER-MEAD VISUALIZATION IN 2D

$$\min_{\mathbf{x}} f(x_1, x_2) = x_1^2 + x_2^2 + x_1 \cdot \sin x_2 + x_2 \cdot \sin x_1$$



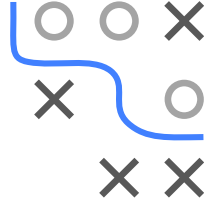
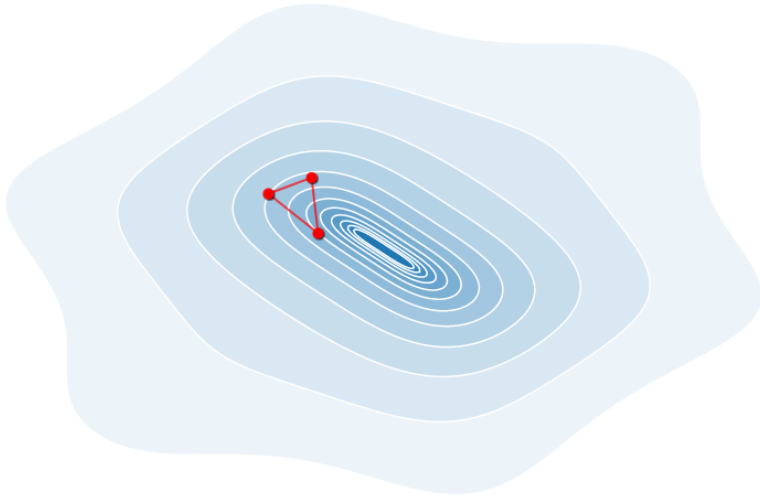
NELDER-MEAD VISUALIZATION IN 2D

$$\min_{\mathbf{x}} f(x_1, x_2) = x_1^2 + x_2^2 + x_1 \cdot \sin x_2 + x_2 \cdot \sin x_1$$



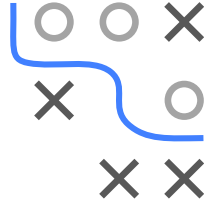
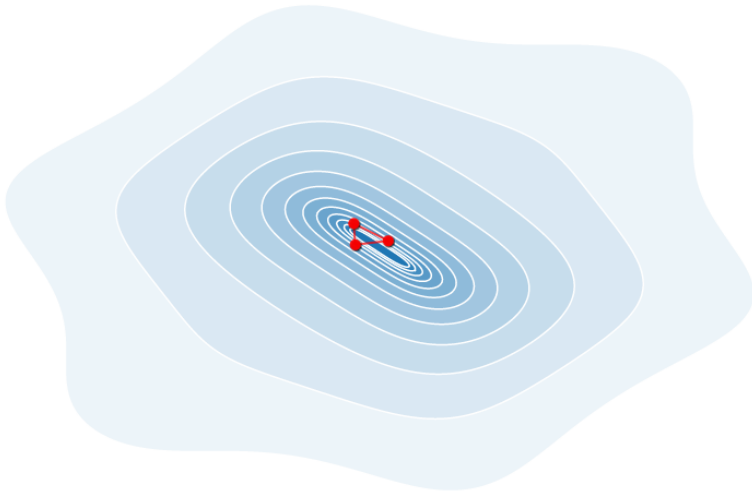
NELDER-MEAD VISUALIZATION IN 2D

$$\min_{\mathbf{x}} f(x_1, x_2) = x_1^2 + x_2^2 + x_1 \cdot \sin x_2 + x_2 \cdot \sin x_1$$

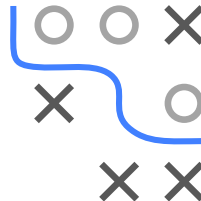
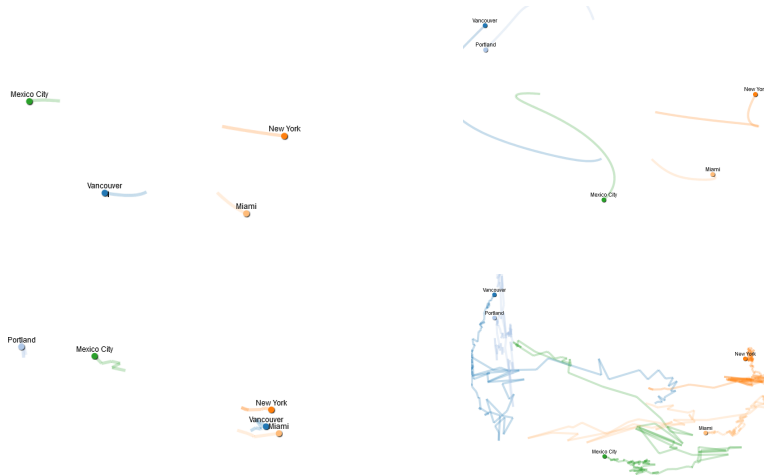


NELDER-MEAD VISUALIZATION IN 2D

$$\min_{\mathbf{x}} f(x_1, x_2) = x_1^2 + x_2^2 + x_1 \cdot \sin x_2 + x_2 \cdot \sin x_1$$



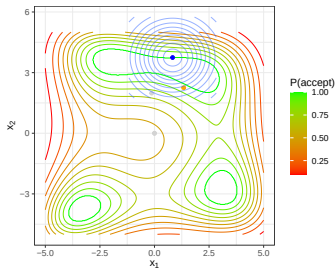
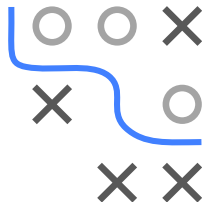
NELDER-MEAD VS. GD



Even for only 5 cities, Nelder-Mead (bottom) performs poorly. However, gradient descent (top) still works.

Optimization in Machine Learning

Simulated Annealing



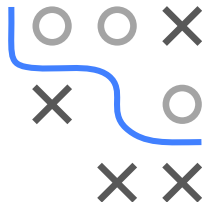
Learning goals

- Motivation
- Metropolis algorithm
- Simulated Annealing

INTRODUCTION

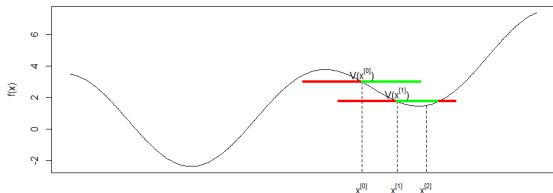
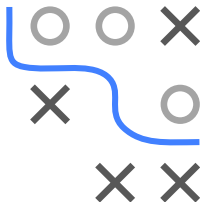
Heuristics for the optimization of complex (multivariate, non-linear, non-convex) objective functions

- Procedure for finding good solutions to complex problems.
- Does not guarantee optimal/best result (global optimum), but usually good solutions.
- Goal for complex optimization problems: avoid “getting stuck” in local optima.
- Is often used for difficult discrete problems as well.
- Local search strategy with random option to accept worse values.



SIMPLE STOCHASTIC LOCAL SEARCH

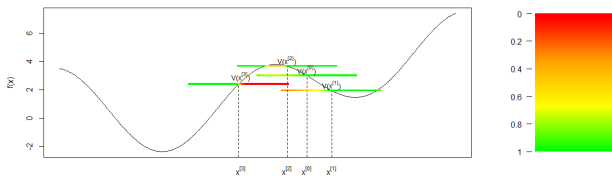
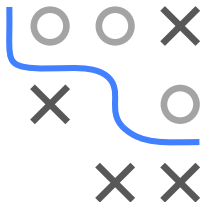
- Given is a multivariate objective function $f(\mathbf{x})$
- Define a local neighborhood area $V(\mathbf{x})$ for a given $\mathbf{x}$
- Sample proposal $\mathbf{x}^{[t+1]}$ uniformly at random from neighborhood $V(\mathbf{x}^{[t]})$
- Calculate $f(\mathbf{x}^{[t+1]})$
- If $\Delta f = f(\mathbf{x}^{[t+1]}) - f(\mathbf{x}^{[t]}) < 0$, $\mathbf{x}^{[t+1]}$ is accepted as new solution, otherwise a new proposal from neighborhood is sampled.



Simple stochastic local search: Acceptance (green) and rejection range (red)

METROPOLIS ALGORITHM

- Simple stochastic local search strongly depends on $\mathbf{x}^{[0]}$ and the neighborhood.
⇒ Danger of ending up in local minima
- **Idea:** allow worse candidates with some probability
- **Metropolis:** accept candidates from previous rejection range ($\Delta f > 0$) with probability $\mathbb{P}(\text{accept} \mid \Delta f) = \exp(-\Delta f / T)$
- T denotes “temperature”

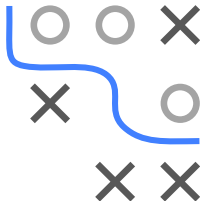
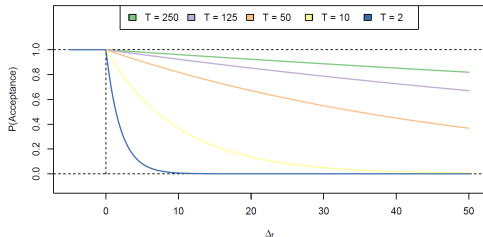


Simulated annealing: Colors correspond to $\mathbb{P}(\text{accept})$

METROPOLIS ALGORITHM

- Parameter T describes temperature/progress of the system
- High temperatures correspond to high probability of accepting worse $\mathbf{x}$
- Local minima can be escaped, but no convergence can be achieved at *constant* temperature
- We come across an important principle of optimization:

exploration (high T) vs. exploitation (low T)



SIMULATED ANNEALING

- Start with high temperature to **explore** whole space
- Slowly reduce temperature to converge
⇒ Sequence of descending temperatures $T^{[t]}, t \in \mathbb{N}$
- Procedure is called **simulated annealing**
- Temperature is often kept constant several iterations in a row to explore the space, then multiplied by coefficient $0 < c < 1$:

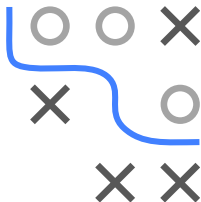
$$T^{[t+1]} = c \cdot T^{[t]}$$

- Other strategies possible, for example:

$$T^{[t]} = T^{[0]} \left(1 - \frac{t}{t_{\max}} \right)$$

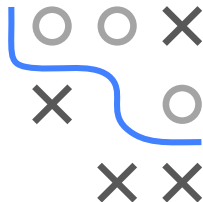
Choosing neighborhood:

- Many different strategies. Strongly depends on objective function.



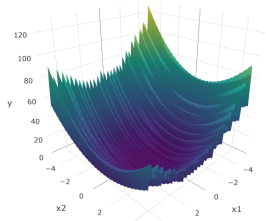
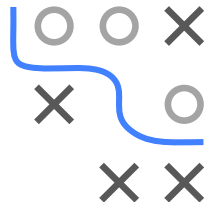
ANALOGY TO METALLURGY

- **Simulated annealing** draws analogy between a cooling process (e.g. a metal or liquid) and an optimization problem.
- If cooling of a liquid material (amount of atoms) is too fast, it solidifies in suboptimal configuration, slow cooling produces crystals with optimal structure (minimum energy stage).
- Consider atoms of the liquid as a system with many degrees of freedom, analogy to optimization problem of a multivariate function
- Minimum energy stage corresponds to optimum of objective function.



Optimization in Machine Learning

Multi-Start Optimization



Learning goals

- Multimodal functions
- Basins of Attractions
- Simple multi-start procedure

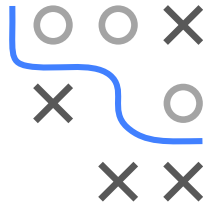
The graph shows a function $f(x)$ defined on the real number line. The function is composed of two parts:

- Blue Curve ($x \leq 0$):** This curve starts at a high value for negative x , decreases to a minimum at x_1 , and then increases to pass through the origin $(0, 0)$.
- Red Curve ($x \geq 0$):** This curve starts at the origin $(0, 0)$, decreases to a minimum at x_2 , and then increases sharply for positive x .

Horizontal black line segments are drawn at $y = -3$ for $x \leq 0$ and $y = -0.5$ for $x \geq 0$.

— x_1
— x_2

- optimum found by Nelder–Mead

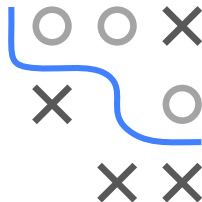
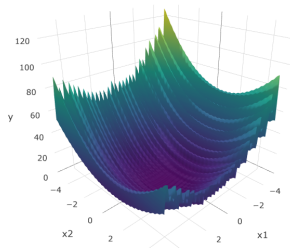


MULTI-STARTS

Levy function:

$$f(\mathbf{x}) = \sin^2(3\pi x_1) + (x_1 - 1)^2[1 + \sin^2(3\pi x_2)] + (x_2 - 1)^2[1 + \sin^2(2\pi x_2)]$$

- Global minimum: $f(\mathbf{x}^*) = 0$ at $\mathbf{x}^* = (1, 1)^\top$
- Optimize f by BFGS method with random starting point in $[-2, 2]^2$ and collect result
- Repeat 100 times



Distribution of results (y values):

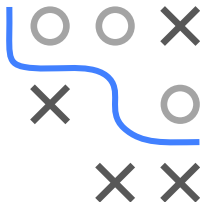
##	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
##	0.0000	0.1099	0.5356	2.4351	1.9809	18.3663

MULTI-STARTS

Idea: use multiple starting points $\mathbf{x}^{[1]}, \dots, \mathbf{x}^{[k]}$ for algorithm A

Algorithm Multistart optimization

- 1: Given: optimization algorithm $A(\cdot)$, $f : \mathcal{S} \mapsto \mathbb{R}$, $\mathbf{x} \mapsto f(\mathbf{x})$
 - 2: $k = 0$
 - 3: **repeat**
 - 4: Draw starting point $\mathbf{x}^{[k]}$ from $\mathcal{S}$ (e.g. uniform if $\mathcal{S}$ is of finite volume)
 - 5: **if** $k = 0$ **then** $\hat{\mathbf{x}} = \mathbf{x}^{[0]}$
 - 6: **end if**
 - 7: Initialize algorithm with start value $\mathbf{x}^{[k]} \Rightarrow \tilde{\mathbf{x}} = A(\mathbf{x}^{[k]})$
 - 8: **if** $f(\tilde{\mathbf{x}}) < f(\hat{\mathbf{x}})$ **then** $\hat{\mathbf{x}} = \tilde{\mathbf{x}}$
 - 9: **end if**
 - 10: $k = k + 1$
 - 11: **until** Stop criterion fulfilled
 - 12: **return** $\hat{\mathbf{x}}$
-



MULTI-STARTS

BFGS with Multistart gives us the true minimum of the Levy function:

```
iters = 20 # number of starts
xbest = c(runif(1, -2, 2), runif(1, -2, 2))

for (i in 1:iters) {
  x1 = runif(1, -2, 2)
  x2 = runif(1, -2, 2)
  res = optim(par = c(x1, x2), fn = f, method = "BFGS")
}

if (res$value < f(xbest)) {
  xbest = res$par
}

xbest
## [1] 1 1
```

